Highlights

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl,Matthias Güdemann

- Machine-checked proof that Crole's five definitions of alpha equivalence coincide.
- Formalized isomorphism of de Bruijn terms and named terms modulo alpha equivalence.
- A roadmap toward a formalized foundation for untyped lambda calculus research.

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl[a,*], Matthias Güdemann[a]

[a] *Munich University of Applied Sciences, Department of Computer Science and Mathematics, Lothstraße 34, Munich, 80335, Bavaria, Germany*

ARTICLE INFO

Keywords:
α-equivalence
permutation action
variable binding
de Bruijn indices
formal verification
proof assistant
Lean

ABSTRACT

Alpha equivalence of lambda terms with named variables has several definitions in the literature: the congruence generated by renaming a bound variable, under one of two side conditions on the new variable, and syntax-directed rules that swap the bound variables of two abstractions with a third one. That these definitions agree is widely accepted, but Crole observed that published proof sketches of this fact are sometimes incorrect, and proved in detail that five such definitions give the same relation. We formalize Crole's theorem in the Lean 4 proof assistant, as part of the library CSLib. To our knowledge the theorem had not been mechanized before. The formalization supplies the proofs that Crole omits or only sketches, and we list earlier sources for the lemmas it needs. As a second result we formalize, following Norrish and Vestergaard, an isomorphism between named terms modulo alpha equivalence and pure de Bruijn terms, with their respective notions of beta-reduction; by Crole's theorem it does not matter which of the five definitions the quotient is taken by. Through the isomorphism, the Church–Rosser theorem for de Bruijn terms becomes a theorem about named terms, and the two representations of lambda terms are connected within one library. Both results are first pieces of a formalized foundation for research on the untyped lambda calculus. The statements in this paper are linked to their Lean sources.

1. Introduction

Lambda terms are identified up to the names of their bound variables. For terms with named variables this identification is a relation on terms, alpha equivalence, and the literature defines it in more than one way. The traditional definition takes the congruence generated by an axiom for renaming a bound variable, where the new variable is required not to occur in the body at all [3, Definition 2.1.11], or only not to occur free in it [10, Definition 1.17]. Gabbay and Pitts [9] instead give syntax-directed rules in which the two bound variables are swapped with a third. That these definitions describe the same relation is widely accepted. Crole [5] proves in detail that five such relations coincide.

The details are easy to get wrong. In the 1988 edition of their textbook, Hindley and Seldin warned that "the above technicalities are rather dull. But their very dullness has made them a trap for even the most careful authors" (quoted in [5, p. 2]). The current edition sketches the proofs about alpha equivalence only, "as they are straightforward and boring" [10, p. 277], and leaves as an exercise that an alpha-contraction cannot always be reversed by a single one, contrary to a claim in [6, p. 91] (see [10, Exercise A1.9]). Crole finds the short proof sketch of Gabbay and Pitts [9, Proposition 2.2] "not quite right", since it lacks a connection between swapping and substitution [5, p. 5]. A formalization that checks every step and stays close to the sources removes this kind of doubt.

Another way to deal with the names of bound variables is to do without them. In de Bruijn's representation a bound variable is a numeral that counts the binders between its occurrence and the binder it refers to. That nameless terms are isomorphic to named terms modulo alpha equivalence is stated, for instance, in [3, Appendix C]. Norrish and Vestergaard [13] give a proof mechanized in HOL4, for pure de Bruijn terms with the β-reduction of Nipkow [12].

*Corresponding author
chris.froeschl@hm.edu (C.A. Fröschl); matthias.guedemann@hm.edu (M. Güdemann)
ORCID(s): 0009-0006-3562-5395 (C.A. Fröschl); 0000-0002-1002-6023 (M. Güdemann)

This paper presents formalizations of both results in the Lean 4 proof assistant. They were developed for CSLib [4], a community-maintained library of computer science for Lean 4 built on Mathlib, whose goal is reviewed and maintained formalizations of standard results on which later research can build directly. Both results extend CSLib's named representation of the untyped lambda calculus, which Section 2 recalls.

1. Section 3 formalizes Crole's theorem. Five relations on named terms, three defined by swapping and two by a renaming axiom, are equal (Theorem 3.4), whereas two further variants of the side condition define different relations (Proposition 3.5). To our knowledge the theorem had not been mechanized before. The formalization follows Crole's proof and supplies the arguments that [5] omits or only sketches (Table 2).
2. Section 4 formalizes the isomorphism of Norrish and Vestergaard between named terms modulo alpha equivalence and de Bruijn terms, with their respective notions of β-reduction (Theorem 4.6). The quotient is taken by the alpha equivalence of CSLib, which is the first of Crole's five relations. As an application, the Church–Rosser theorem for de Bruijn terms yields the Church–Rosser theorem for named terms modulo alpha equivalence (Corollary 4.17). This result is not new as a mechanization. Its role here is to connect the two representations of lambda terms within one library.

The statements of Sections 2 to 4 carry a raised LEAN that links to the corresponding declaration in the Lean sources. A blueprint[1] presents both formalizations together, with a dependency graph[2] that shows how they are connected.

For Section 3, Aristotle was used during the exploration of alternative definitions and auxiliary swap lemmas; the final proofs were substantially rewritten, manually verified, and checked against the original paper. The same holds for the alternative definitions explored for Section 4, where AI assistance was also used to compare the formalization with the HOL4 development behind [13].

Both results are meant as parts of a library on which further work on the untyped lambda calculus can build. Section 5 discusses related work, Section 6 describes the directions we have in mind, in particular the remaining formalizations of the network of Norrish and Vestergaard [13, Figure 1], and Section 7 concludes.

2. Named lambda terms in CSLib

CSLib currently represents untyped lambda terms in a standard named style: variables are drawn from an infinite set of atoms, and terms are built from variables, application, and abstraction. Our notation follows Crole [5], except that we write the concrete syntax of lambda terms where Crole writes $P(E_1, E_2)$ and $B([a]E)$ for a generic binary constructor and a generic binder; Crole notes that his proofs are insensitive to this choice [5, Section 2].

Definition 2.1 (Named lambda terms)**.** LEAN Let $\mathbb{A}$ be a set of atoms. The set Λ of named lambda terms is generated by

$$M \ ::= \ a \mid M\,M \mid \lambda a.\, M \qquad (a \in \mathbb{A}).$$

We write $\mathrm{occ}(M)$ LEAN for the finite set of all atoms occurring in M, including binding occurrences, and $\mathrm{free}(M)$ LEAN for the finite set of atoms occurring free in M, and abbreviate

$$a \not\triangleright M \iff a \notin \mathrm{occ}(M) \qquad \text{and} \qquad a \# M \iff a \notin \mathrm{free}(M).$$

In the second case we say that a is *fresh* for M. Both notations extend to lists, so that $z \not\triangleright a, b, M, M'$ means that z differs from a and b and occurs in neither M nor M'.

Crole assumes $\mathbb{A}$ to be countable, with equality of atoms as its only operation. In CSLib, $\mathbb{A}$ is an arbitrary type with decidable equality that comes with a function fresh from finite sets of atoms to atoms such that $\mathrm{fresh}(S) \notin S$; in particular $\mathbb{A}$ is infinite, and an atom avoiding any given finite set can be computed.

Three operations act on the atoms of a term. The first is purely textual and is the one CSLib's baseline alpha equivalence is phrased with.

[1] https://chris-anto-froeschl.github.io/untyped-lambda-calculus/
[2] https://chris-anto-froeschl.github.io/untyped-lambda-calculus/blueprint/dep_graph_document.html

Definition 2.2 (Renaming). LEAN $M\langle a \mapsto b\rangle$ is M with every occurrence of a, free, bound or binding, replaced by b.

The second applies a bijection on atoms to every atom of a term, binding occurrences included.

Definition 2.3 (Permutation action [5, Section 2]). LEAN For a permutation π of $\mathbb{A}$, the term $\pi \cdot M$ is defined by

$$\pi \cdot a = \pi(a) \qquad \pi \cdot (M_1\, M_2) = (\pi \cdot M_1)\,(\pi \cdot M_2) \qquad \pi \cdot (\lambda a.\, M) = \lambda \pi(a).\, \pi \cdot M.$$

For the transposition $(a\ b)$ we call $(a\ b) \cdot M$ the *swap* LEAN of a and b in M. The *agreement set* of two permutations is $\mathrm{AS}(\pi, \pi') = \{\, a \in \mathbb{A} \mid \pi(a) = \pi'(a) \,\}$.

Unlike renaming (Definition 2.2), a swap is a bijection on terms and never identifies two atoms. The two operations agree when the new atom is unused: if $z \notin M$ then $(z\ a) \cdot M = M\langle a \mapsto z\rangle$ LEAN.

The third is capture-avoiding substitution, which Crole writes $E\{a'/a\}$ for the substitution of an atom and which we write $M[a := N]$ for the substitution of an arbitrary term.

Definition 2.4 (Capture-avoiding substitution). LEAN

$$b[a := N] = \begin{cases} N & \text{if } b = a \\ b & \text{otherwise} \end{cases}$$

$$(M_1\, M_2)[a := N] = M_1[a := N]\ M_2[a := N]$$

$$(\lambda b.\, M)[a := N] = \begin{cases} \lambda b.\, M & \text{if } b = a \\ \lambda b.\, M[a := N] & \text{if } b \neq a \text{ and } b \# N \\ \lambda c.\, M\langle b \mapsto c\rangle[a := N] & \text{otherwise, where } c = \mathrm{fresh}(\mathrm{occ}(M) \cup \mathrm{occ}(N) \cup \{a, b\}) \end{cases}$$

Example 2.5 (The three operations). Let $M = \lambda a.\, a\, b$ for distinct atoms a and b.

- $M\langle a \mapsto b\rangle = \lambda b.\, b\, b$ LEAN: renaming identifies the two atoms, so that the free b is captured.
- $(a\ b) \cdot M = \lambda b.\, b\, a$ LEAN: the swap exchanges them.
- $M[b := a] = \lambda c.\, c\, a$ for a fresh atom c LEAN: substitution replaces only the free occurrence of b, and renames the binder to avoid capturing a.

Definition 2.4 is, case for case, Gabbay's [7, Definition 5.13]: a function chooses the new atom fresh for a, b and the atoms of M and N, and the proof that the definition is well founded is his induction on the size of the term, which renaming preserves [7, Lemma 5.14]. Gabbay renames the binder with a substitution $M[b \mapsto c]$, which coincides with $M\langle b \mapsto c\rangle$ since c is fresh. Like Curry's substitution [3, Definition C.1], it is defined on all terms, but it renames b whenever b is free in N, whereas Curry keeps b also when a is not free in M. Renaming and substitution, a notion of alpha equivalence (Definition 3.1 below) and its basic theory, including the substitution lemma [3, 2.1.16] LEAN, are due to Yin and Montesi[3] and are the starting point of our work: the alpha equivalence of CSLib is the reference point against which the other definitions are shown equivalent in Section 3.

3. Characterizations of alpha equivalence

This section presents Crole's result [5] as it is formalized in CSLib. We keep Crole's names for the relations, the rules and the side conditions, and each statement names its counterpart in [5].

[3] https://github.com/leanprover/cslib/pull/458

3.1. The five relations

The first three relations are syntax-directed and differ only in the side condition on the atom z that replaces the two bound atoms.

Definition 3.1 ([5, Definitions 3.1–3.3]). The relation $\sim_{\mathrm{p}}$ LMN on Λ is inductively defined by

$$\frac{}{a \sim_{\mathrm{p}} a}\ \textsc{atom} \qquad \frac{M_1 \sim_{\mathrm{p}} M_1' \quad M_2 \sim_{\mathrm{p}} M_2'}{M_1\, M_2 \sim_{\mathrm{p}} M_1'\, M_2'}\ \textsc{pcg} \qquad \frac{(z\ a) \cdot M \sim_{\mathrm{p}} (z\ b) \cdot M'}{\lambda a.\, M \sim_{\mathrm{p}} \lambda b.\, M'}\ \textsc{pi}\ [z \fresh a, b, M, M'].$$

The relations $\sim_{\mathrm{p}\#}$ LMN and $\sim_{\mathrm{p}}^{1}$ LMN are defined by the same rules, with the side condition of PI replaced by

$$[z \mathbin{\#} a, b, M, M'] \qquad \text{and} \qquad [z \fresh M, M'],$$

respectively; we call the resulting rules PI# and PI1.

The relation $\sim_{\mathrm{p}}$ follows Gabbay and Pitts [9] and is CSLib's alpha equivalence $=_\alpha$. The remaining two relations are generated by the renaming axiom and are congruences by definition.

Definition 3.2 ([5, Definitions 3.4 and 3.5]). The relation $\sim_{\mathrm{r}}$ LMN on Λ is inductively defined by

$$\frac{}{M \sim_{\mathrm{r}} M}\ \textsc{ref} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2}{M_2 \sim_{\mathrm{r}} M_1}\ \textsc{sym} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2 \quad M_2 \sim_{\mathrm{r}} M_3}{M_1 \sim_{\mathrm{r}} M_3}\ \textsc{trs}$$

$$\frac{M_1 \sim_{\mathrm{r}} M_1' \quad M_2 \sim_{\mathrm{r}} M_2'}{M_1\, M_2 \sim_{\mathrm{r}} M_1'\, M_2'}\ \textsc{pcg} \qquad \frac{M \sim_{\mathrm{r}} M'}{\lambda a.\, M \sim_{\mathrm{r}} \lambda a.\, M'}\ \textsc{bcg} \qquad \frac{}{\lambda a.\, M \sim_{\mathrm{r}} \lambda a'.\, M[a := a']}\ \alpha\ [a' \fresh a, M].$$

The relation $\sim_{\mathrm{r}\#}$ LMN is defined by the same rules, with the side condition of α replaced by $[a' \mathbin{\#} a, M]$; we call the resulting rule $\alpha\#$.

The relation $\sim_{\mathrm{r}}$ corresponds to the congruence $\equiv_\alpha$ of Barendregt [3, Definition 2.1.11], and $\sim_{\mathrm{r}\#}$ to that of Hindley and Seldin [10, Definition 1.17].

Example 3.3 (Running example). For distinct atoms a, b, c, the terms $\lambda a.\, a\, b$ and $\lambda c.\, c\, b$ are related by all five relations, each time by a single application of the abstraction rule.

- PI LMN, PI# LMN and PI1 LMN: any witness d distinct from a, b, c meets all three side conditions, and $(d\ a) \cdot (a\ b) = d\, b = (d\ c) \cdot (c\ b)$.
- α LMN and $\alpha\#$ LMN: rename a to c, which does not occur in $a\, b$; then $(a\, b)[a := c] = c\, b$.

3.2. The characterization theorem

Theorem 3.4 (Crole [5, Theorems 4.1, 4.2, 4.4–4.6]). *For all terms $M, N \in \Lambda$ the following are equivalent:*

(i) $M \sim_{\mathrm{p}} N$;
(ii) $M \sim_{\mathrm{p}\#} N$ LMN;
(iii) $M \sim_{\mathrm{p}}^{1} N$ LMN;
(iv) $M \sim_{\mathrm{r}} N$ LMN;
(v) $M \sim_{\mathrm{r}\#} N$ LMN.

The proof is outlined in Section 3.3. The side condition of PI cannot be weakened much further.

Proposition 3.5 ([5, Proposition 4.3]). LMN *Let $\sim_{\mathrm{p}}^{2}$ LMN and $\sim_{\mathrm{p}}^{3}$ LMN be defined as $\sim_{\mathrm{p}}$, with the side condition of PI replaced by $[z \fresh a, b]$ and by no condition, respectively. Then $\sim_{\mathrm{p}}^{2} \neq \sim_{\mathrm{p}}$ and $\sim_{\mathrm{p}}^{3} \neq \sim_{\mathrm{p}}$.*

Proof. For distinct atoms a, b, z we have $\lambda a.\, a\, z \sim_{\mathrm{p}}^{2} \lambda b.\, b\, a$ LMN, with witness z, and $\lambda a.\, b\, a \sim_{\mathrm{p}}^{3} \lambda b.\, a\, b$ LMN, with witness b, but in both cases the two terms have different free atoms, so they are not related by $\sim_{\mathrm{p}}$, which preserves free atoms LMN LMN LMN. □

Table 1
The cases of the rule PI in the proof of Lemma 3.6, by the values of a, b, z (o: neither u nor v). Entries 1–4 are Crole's representative cases [5, pp. 14–15], with new witness z, v, u, u respectively: 1 is (o, o, o), 2 is (o, o, u), 3 is (o, u, v), 4 is (u, u, v). A superscript σ means that the case follows by exchanging u and v, and τ by exchanging the two sides of $\sim_{\mathrm{p}}$ (symmetry); $\times$ marks the cases excluded by $z \neq a, b$. The assignment is the one of the Lean proof.

	$z = u$			$z = v$			$z = o$		
	$b = u$	$b = v$	$b = o$	$b = u$	$b = v$	$b = o$	$b = u$	$b = v$	$b = o$
$a = u$	$\times$	$\times$	$\times$	4	$\times$	3^{τ}	1	1	1
$a = v$	$\times$	4^{σ}	$3^{\sigma\tau}$	$\times$	$\times$	$\times$	1	1	1
$a = o$	$\times$	3^{σ}	2	3	$\times$	2^{σ}	1	1	1

3.3. The proof

Since $\sim_{\mathrm{p}}$ has no rules for reflexivity, symmetry and transitivity, these have to be proved. CSLib already provides them [LEAN LEAN LEAN], together with two facts used below: related terms have the same free atoms [LEAN], and substitution respects $\sim_{\mathrm{p}}$: if $M \sim_{\mathrm{p}} M'$ and $N \sim_{\mathrm{p}} N'$ then $M[a := N] \sim_{\mathrm{p}} M'[a := N']$ [LEAN]. The corresponding statements for $\equiv_{\alpha}$ are [10, Lemmas 1.19 and 1.21], proved in [10, Appendix A1]. Beyond these, the proof of Theorem 3.4 rests on three facts: $\sim_{\mathrm{p}}$ is preserved by swapping, a permutation acts up to $\sim_{\mathrm{p}}$ only through the free atoms, and swapping with an unused atom is, up to alpha equivalence, substitution.

Lemma 3.6 ([5, Lemma 6.1]). [LEAN] *If $M \sim_{\mathrm{p}} M'$ then $(u\ v) \cdot M \sim_{\mathrm{p}} (u\ v) \cdot M'$, for all atoms u, v.*

This is an instance of the equivariance principle of nominal techniques [7, Theorem 4.4]. For the rule PI, Crole argues four representative cases in detail [5, pp. 14–15]; the formalization proves all 17 possible cases, by these four arguments and two symmetries (Table 1).

Lemma 3.7 ([5, Lemma 6.2]). *Let π, π' be permutations of $\mathbb{A}$ and M a term.*

1. [LEAN] *If $\mathrm{occ}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M = \pi' \cdot M$.*
2. [LEAN] *If $\mathrm{free}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M \sim_{\mathrm{p}} \pi' \cdot M$.*

The third fact links the two families of relations. For the renaming relations it is [5, Lemmas 6.3 and 6.4]. Crole observes that Gabbay and Pitts need it for their proof as well, where they call it and Lemma 3.6 easily verified [9, proof of Proposition 2.2], and that it is missing from their sketch [5, p. 5].

Lemma 3.8. *If $z \not\triangleright M$ then $(z\ a) \cdot M \sim_{\mathrm{r}} M[a := z]$ [LEAN] and $(z\ a) \cdot M \sim_{\mathrm{r}\#} M[a := z]$ [LEAN].*

Example 3.9. The two sides of Lemma 3.8 differ on bound occurrences of a, which the swap replaces and the substitution does not. Let $M = a\,(\lambda a.\, a)$ and $z \neq a$. Then

$$(z\ a) \cdot M = z\,(\lambda z.\, z) \qquad \text{and} \qquad M[a := z] = z\,(\lambda a.\, a)$$

[LEAN LEAN], and the two are related by PCG, α and SYM [LEAN].

For $\sim_{\mathrm{p}}$ the corresponding statement compares two swaps. We need only the case $n = 1$ of Crole's lemma.

Lemma 3.10 ([5, Lemma 6.5], case $n = 1$). [LEAN] *Let z, a, a' be distinct atoms and M a term with $z \# M$ and $a' \# M$. Then $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ a') \cdot M[a := a']$.*

Proof idea. Crole omits the proof. First $M[a := a'] \sim_{\mathrm{p}} (a\ a') \cdot M$: rename the bound occurrences of a' away [5, Lemma 6.6], use that substitution respects $\sim_{\mathrm{p}}$, and apply the $\sim_{\mathrm{p}}$-form of Lemma 3.8. Then $(z\ a)$ and $(z\ a') \circ (a\ a')$ agree on $\mathrm{free}(M)$, and Lemma 3.7 applies. □

Crole states the lemma for n swaps applied one after the other. The formalization proves this general form, of which Lemma 3.10 is the case $n = 1$, by an induction on n that commutes the remaining swaps past the first.

Proof idea of Theorem 3.4. Each of (ii)–(v) is shown equivalent to (i), by showing that each of the two relations is closed under the rules of the other. Only the rules for abstractions need an argument, and except for the rules α and $\alpha\#$ it is Crole's [5, pp. 5–8]. A witness that is only fresh (PI#) or may equal a bound atom (PI1) is exchanged for an unused one by Lemmas 3.6 and 3.7. The renaming relations are closed under PI by Lemma 3.8, which turns the two swaps of its premise into substitutions, and two instances of α or $\alpha\#$.

For the converse, reflexivity, symmetry and transitivity of $\sim_{\mathrm{p}}$ are already in CSLib, and BCG follows from Lemma 3.6 and PI. For α and $\alpha\#$, let $a' \# M$ and choose $z \not\in a, a', M, M[a := a']$. Then $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ a') \cdot M[a := a']$ by Lemma 3.10, which is the premise of PI for $\lambda a.\, M \sim_{\mathrm{p}} \lambda a'.\, M[a := a']$. □

3.4. Notes on the formalization

The formalization follows [5] closely, and we found no errors in it. Our contribution is the four relations other than $\sim_{\mathrm{p}}$, Theorem 3.4, Proposition 3.5, Lemmas 3.6–3.10 and Crole's Lemma 6.6. The rest belongs to the named representation of Section 2. The first part of the development is under review for inclusion in CSLib[4]; the remainder is to follow in the same way.

Where the formalization departs from [5] beyond the proofs discussed above:

- *Order of the results.* Reflexivity, symmetry and transitivity of $\sim_{\mathrm{p}}$ are proved directly. Crole obtains reflexivity, symmetry and transitivity of $\sim_{\mathrm{p}}$ inside the proof of his Theorem 4.4 [5, p. 6] and cites that theorem for them in the proof of Theorem 4.1 [5, p. 5]. In the formalization Theorem 4.1 does not depend on Theorem 4.4, which is what allows the split into pull requests.

- *Renaming in PI.* The rules PI and PI1 are stated in CSLib with renaming (Definition 2.2) in place of the swap, which is the same under their side conditions.

- *The rules α and $\alpha\#$.* Crole proves α by an induction on M [5, pp. 6–7] that fails for $\alpha\#$, where $M[a := a']$ may have to rename a binder, and introduces Lemma 6.5 for general n to repair it [5, p. 8]. The formalization derives both rules from the case $n = 1$ (Lemma 3.10).

- *Freshness in Lemma 6.5.* Crole assumes $z \not\in M$; freshness suffices, because a bound occurrence of z can be renamed away first by Crole's Lemma 6.6: if $a \# M$ then there is a term $\hat{M}$ with $a \not\in \hat{M}$ and $\hat{M} \sim_{\mathrm{r}} M$ LEAN. This lemma justifies Barendregt's variable convention [3, 2.1.13].

- *Lemma 6.6 for $\sim_{\mathrm{p}}$.* Crole states Lemma 6.6 for $\sim_{\mathrm{r}}$ only. Lemma 3.10 needs it for $\sim_{\mathrm{p}}$ LEAN, and this form cannot be derived through Theorem 3.4, whose proof uses Lemma 3.10. The proof follows Crole's sketch [5, p. 16], except where the binder is a itself: renaming a to an unused atom throughout the body removes every occurrence of a at once, so that no induction on the size of the term is needed.

- *Permutations.* Lemma 3.7 is proved for arbitrary bijections on $\mathbb{A}$, where Crole assumes finite support; the difference is immaterial for terms [7, Remark 2.25]. Agreement on a set of atoms is stated with Mathlib's `Set.EqOn` instead of the agreement set.

- *Scope.* Crole's second contribution, alpha equivalence of program contexts [5, Section 5], is not covered.

What a formalization has to add to [5] are the proofs that the paper leaves to the reader or to the literature. Table 2 lists the lemmas of [5, Section 6.1] that the characterization needs, how much of their proofs Crole gives, and where the statement or its argument appears earlier in the literature. Lemma 6.7, which Crole uses only for a second, direct proof of Theorem 4.6, is not needed and not formalized. Crole names a source only for Lemma 6.3; the other entries in the last column are ours. None of the omitted proofs held a surprise.

4. An isomorphism with de Bruijn terms

De Bruijn indices remove the need for alpha equivalence altogether by replacing bound variable names with numerals counting binder depth. Norrish and Vestergaard [13] give a direct, mechanized proof that the

[4] `https://github.com/leanprover/cslib/pull/741`

Table 2
The lemmas of [5, Section 6.1]: their proofs in [5] and earlier sources.

In [5]	Proof there	Earlier source
Lemma 6.1	representative cases	[9, proof of Proposition 2.2]; [16, (9)]; [7, Theorem 4.4]
Lemma 6.2, part 1	omitted	[9, Example 3.5(ii)]
Lemma 6.2, part 2	given	[16, Lemma 2.8]; [8, Lemma 3.8]
Lemma 6.3	given	[7, Lemma 5.16]; [8, Lemma 3.6(2)]
Lemma 6.4	as for Lemma 6.3	as for Lemma 6.3
Lemma 6.5	details omitted	none for general n; for $n = 1$ our argument combines [9, proof of Proposition 2.2] and [10, Lemmas A1.7, A1.8]
Lemma 6.6	sketched, well known	[10, Lemma A1.5(g)]

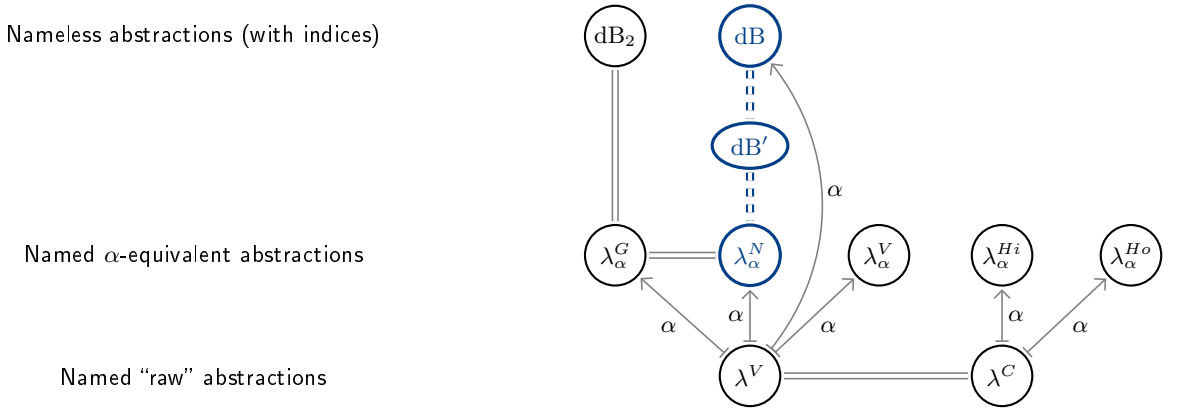


Figure 1: Formalizations of the untyped lambda calculus and their directly proved relationships, redrawn from [13, Figure 1] with the help of Claude (Anthropic) and checked against the original. Double lines are isomorphisms and arrows are α-quotients. The dashed lines are the results of [13], which this section formalizes; thick blue outlines mark what is formalized in CSLib.

pure de Bruijn calculus is isomorphic to the alpha-quotiented named calculus, so that a result proved on de Bruijn terms is a result about lambda terms in the ordinary sense. We formalize this isomorphism for the named terms and the de Bruijn terms of CSLib; the formalization is under review for inclusion in CSLib[5], on top of the pull requests for the de Bruijn terms (below) and for Theorem 4.1 of Section 3. By Theorem 3.4 it does not matter which of the five alpha equivalences is quotiented by; the formalization uses $\sim_{\mathrm{p}}$.

Norrish and Vestergaard place their isomorphism in a web of directly proved relationships between ten formalizations of the untyped lambda calculus, reproduced in Figure 1. With this section, CSLib contains the nodes dB, dB′ and λ^N_α and the two dashed edges between them.

We keep the notation of [13] on the de Bruijn side, where t, u, s range over de Bruijn terms and i, j, k, n over indices. As in [13], M and N range over alpha-equivalence classes in this section.

4.1. De Bruijn terms

The de Bruijn terms, their substitution and β-reduction, and the proof of the Church–Rosser theorem used in Section 4.7 are not ours: they are due to Wang, the syntax and substitution jointly with Radziwill[6].

Definition 4.1 (De Bruijn terms [13, Section 2.2]). LEAN The set dB of de Bruijn terms is generated by

$$t ::= \mathsf{dV}\, i \mid \mathsf{dAPP}\, t\, t \mid \mathsf{dABS}\, t \qquad (i \in \mathbb{N}).$$

[5] https://github.com/leanprover/cslib/pull/TODO
[6] https://github.com/leanprover/cslib/pull/475

Lifting and substitution are defined as in Nipkow's formalization [12]:

$$\begin{aligned}
\mathsf{lift}\,(\mathsf{dV}\,i)\,k &= \text{if } i < k \text{ then } \mathsf{dV}\,i \text{ else } \mathsf{dV}\,(i+1)\\
\mathsf{lift}\,(\mathsf{dAPP}\,t\,u)\,k &= \mathsf{dAPP}\,(\mathsf{lift}\,t\,k)\,(\mathsf{lift}\,u\,k)\\
\mathsf{lift}\,(\mathsf{dABS}\,t)\,k &= \mathsf{dABS}\,(\mathsf{lift}\,t\,(k+1))
\end{aligned}$$

$$\begin{aligned}
\mathsf{nsub}\,s\,k\,(\mathsf{dV}\,i) &= \text{if } k < i \text{ then } \mathsf{dV}\,(i-1) \text{ else if } i = k \text{ then } s \text{ else } \mathsf{dV}\,i\\
\mathsf{nsub}\,s\,k\,(\mathsf{dAPP}\,t\,u) &= \mathsf{dAPP}\,(\mathsf{nsub}\,s\,k\,t)\,(\mathsf{nsub}\,s\,k\,u)\\
\mathsf{nsub}\,s\,k\,(\mathsf{dABS}\,t) &= \mathsf{dABS}\,(\mathsf{nsub}\,(\mathsf{lift}\,s\,0)\,(k+1)\,t)
\end{aligned}$$

The function nsub substitutes for the index k and at the same time decrements all larger free indices, which is what β-contraction needs when a binder disappears.

Definition 4.2 ([13, Definition 1]). The relation $\to_{\mathrm{d}}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\,(\mathsf{dABS}\,t)\,u \to_{\mathrm{d}} \mathsf{nsub}\,u\,0\,t} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dABS}\,t \to_{\mathrm{d}} \mathsf{dABS}\,u}$$

$$\frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\,t\,s \to_{\mathrm{d}} \mathsf{dAPP}\,u\,s} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\,s\,t \to_{\mathrm{d}} \mathsf{dAPP}\,s\,u}\,.$$

4.2. The alpha-quotiented calculus

Definition 4.3 ([13, Section 2.1]). Λ_α is the quotient of Λ by $\sim_{\mathrm{p}}$. We write $[M]_\alpha$ for the class of a term $M \in \Lambda$. Application, abstraction and substitution are lifted from Λ to Λ_α:

$$[M]_\alpha\,[N]_\alpha = [M\,N]_\alpha \qquad \lambda a.\,[M]_\alpha = [\lambda a.\,M]_\alpha \qquad [M]_\alpha[a := [N]_\alpha] = [M[a := N]]_\alpha.$$

These operations are well defined because $\sim_{\mathrm{p}}$ is a congruence and is preserved by substitution. Abstraction on Λ_α is not injective (Example 4.4), so Λ_α is not a free algebra.

Example 4.4. For distinct atoms a, b, c, $\lambda a.\,[a\,b]_\alpha = \lambda c.\,[c\,b]_\alpha$ (Example 3.3), although $[a\,b]_\alpha \neq [c\,b]_\alpha$.

Definition 4.5 (β-reduction on Λ_α). The relation $\to_\beta$ on Λ_α is the least relation satisfying

$$\frac{}{(\lambda a.\,M)\,N \to_\beta M[a := N]} \qquad \frac{M \to_\beta N}{\lambda a.\,M \to_\beta \lambda a.\,N}$$

$$\frac{M \to_\beta N}{M\,Z \to_\beta N\,Z} \qquad \frac{M \to_\beta N}{Z\,M \to_\beta Z\,N}\,.$$

This is the system $\lambda^{\mathrm{N}}_\alpha$ of [13]: substitution and reduction are defined on the quotient itself and not on raw terms.

Two systems $\langle X, \to_X\rangle$ and $\langle Y, \to_Y\rangle$ are *isomorphic* if there is a bijection $f\colon X \to Y$ with $x_1 \to_X x_2 \iff f(x_1) \to_Y f(x_2)$ for all x_1, x_2 [13, Section 2.3].

4.3. The isomorphism theorem

Theorem 4.6 ([13, Theorem 20]). *The systems $\langle \Lambda_\alpha, \to_\beta\rangle$ and $\langle \mathrm{dB}, \to_{\mathrm{d}}\rangle$ are isomorphic, as witnessed by the bijection f of Theorem 4.10 below: for all $M, N \in \Lambda_\alpha$,*

$$M \to_\beta N \iff f(M) \to_{\mathrm{d}} f(N).$$

Proof. From Theorems 4.12 and 4.15 below. □

The notion of isomorphic reduction systems used here is exactly Mathlib's relation isomorphism, and the formalization also states Theorem 4.6 in that form, $\langle \Lambda_\alpha, \to_\beta\rangle \simeq_r \langle \mathrm{dB}, \to_{\mathrm{d}}\rangle$.

The proof of Norrish and Vestergaard goes through an intermediate formalism (Figure 2). Section 4.4 gives a second set of rules on the de Bruijn terms that is built to resemble Λ_α. Section 4.5 constructs f and shows that it turns $\to_\beta$ into these rules, and Section 4.6 shows that they define the same relation as $\to_{\mathrm{d}}$.

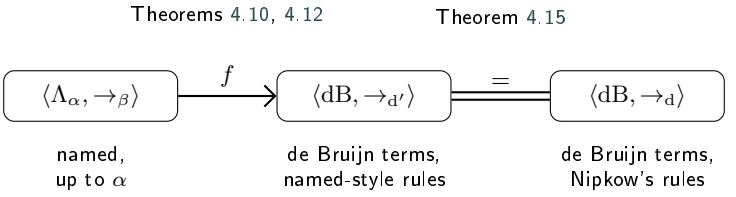


Figure 2: The route of the proof of Theorem 4.6.

4.4. An intermediate formalism

The proof goes through a second presentation dB' of the same type dB that is built to resemble Λ_α. It replaces nsub by a substitution that leaves the other indices alone, and dABS by an abstraction over a chosen index.

Definition 4.7 ([13, Definitions 4 and 5]). LEMMA LEMMA

$$\begin{aligned}
\mathsf{sub}\, s\, k\, (\mathsf{dV}\, i) &= \text{if } i = k \text{ then } s \text{ else } \mathsf{dV}\, i \\
\mathsf{sub}\, s\, k\, (\mathsf{dAPP}\, t\, u) &= \mathsf{dAPP}\, (\mathsf{sub}\, s\, k\, t)\, (\mathsf{sub}\, s\, k\, u) \\
\mathsf{sub}\, s\, k\, (\mathsf{dABS}\, t) &= \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{lift}\, s\, 0)\, (k+1)\, t) \\
\mathsf{dLAM}\, i\, t &= \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{dV}\, 0)\, (i+1)\, (\mathsf{lift}\, t\, 0))
\end{aligned}$$

Like abstraction on Λ_α, dLAM is not injective (Example 4.8). Every dABS term is a dLAM term over any index that is not free in it LEMMA [13, Lemma 6].

Example 4.8. LEMMA For every index i,

$$\mathsf{dLAM}\, i\, (\mathsf{dV}\, i) = \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{dV}\, 0)\, (i+1)\, (\mathsf{dV}\, (i+1))) = \mathsf{dABS}\, (\mathsf{dV}\, 0),$$

the de Bruijn form of $\lambda a.\, a$ for every atom a.

Definition 4.9 ([13, Definition 7]). LEMMA The relation $\to_{\mathrm{d}'}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\, (\mathsf{dLAM}\, i\, t)\, u \to_{\mathrm{d}'} \mathsf{sub}\, u\, i\, t} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dLAM}\, i\, t \to_{\mathrm{d}'} \mathsf{dLAM}\, i\, u}$$

$$\frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\, t\, s \to_{\mathrm{d}'} \mathsf{dAPP}\, u\, s} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\, s\, t \to_{\mathrm{d}'} \mathsf{dAPP}\, s\, u}\,.$$

4.5. The bijection

Fix a bijection between $\mathbb{A}$ and $\mathbb{N}$, written with a hat in both directions as in [13]: $\hat{a}$ is the number of the atom a and $\hat{\imath}$ the atom with number i.

Theorem 4.10 ([13, Theorem 11]). LEMMA *There is a bijection* $f\colon \Lambda_\alpha \to \mathrm{dB}$ *with*

$$f([a]_\alpha) = \mathsf{dV}\, \hat{a} \qquad f(M\, N) = \mathsf{dAPP}\, (f\, M)\, (f\, N) \qquad f(\lambda a.\, M) = \mathsf{dLAM}\, \hat{a}\, (f\, M). \tag{1}$$

Example 4.11 (Running example, continued). LEMMA For distinct atoms a and b,

$$f(\lambda a.\, a\, b) = \mathsf{dLAM}\, \hat{a}\, (\mathsf{dAPP}\, (\mathsf{dV}\, \hat{a})\, (\mathsf{dV}\, \hat{b})) = \mathsf{dABS}\, (\mathsf{dAPP}\, (\mathsf{dV}\, 0)\, (\mathsf{dV}\, (\hat{b}+1))) :$$

the bound atom becomes the index 0, and the free atom b is shifted past the binder. For every atom $c \neq b$, the alpha-equivalent term $\lambda c.\, c\, b$ has the same image LEMMA.

Since Λ_α is not a free algebra, the equations (1) do not define a function by themselves. Norrish and Vestergaard obtain f from Pitts' recursion principle for Λ_α [14], for which dB has to be shown to be a nominal set. The formalization takes a more elementary route: the equations (1) define a function on raw terms by structural recursion, this function is shown to respect $\sim_\mathrm{p}$, and f is its lift to the quotient. Injectivity is proved by structural induction on raw terms. Its abstraction case rests on the fact that dLAM is injective up to a transposition: $\mathsf{dLAM}\,i\,t = \mathsf{dLAM}\,j\,u$ holds exactly if $i = j$ and $t = u$, or $i \neq j$, the index i is not free in u, and t is u with i and j exchanged. Surjectivity is proved, as in [13], by complete induction on the size of the de Bruijn term, using Lemma 6 of [13] for dABS terms.

Theorem 4.12 ([13, Theorem 12])**.** *For all $M, N \in \Lambda_\alpha$: $M \to_\beta N$ if and only if $f(M) \to_{\mathrm{d}'} f(N)$.*

Proof idea. The bijection turns substitution on Λ_α into the index-preserving substitution sub, $f(M[a := N]) = \mathsf{sub}\,(f\,N)\,\hat{a}\,(f\,M)$, and abstraction into dLAM, so that each rule of $\to_\beta$ becomes the corresponding rule of $\to_{\mathrm{d}'}$ [13, proof of Theorem 12]. □

4.6. From $\to_{\mathrm{d}'}$ to $\to_\mathrm{d}$

It remains to show that $\to_\mathrm{d}$ and $\to_{\mathrm{d}'}$ are the same relation on dB. Their rules for redexes are related by an identity between sub and nsub [13, Lemma 13]; the difficulty is to match the congruence rule for dABS with the one for dLAM. This is where permutations enter on the de Bruijn side. A permutation π of atoms acts on a de Bruijn term through the free indices:

$$\pi \cdot (\mathsf{dV}\,i) = \mathsf{dV}\,\widehat{\pi}(i) \qquad \pi \cdot (\mathsf{dAPP}\,t\,u) = \mathsf{dAPP}\,(\pi \cdot t)\,(\pi \cdot u) \qquad \pi \cdot (\mathsf{dABS}\,t) = \mathsf{dABS}\,(\mathsf{inc}(\pi) \cdot t),$$

where $\mathsf{inc}(\pi)$ is π shifted by one index, so that it fixes the index bound by the abstraction [13, Definition 8].

Example 4.13. Let $\hat{a} = 0$ and $\hat{b} = 1$. The term $\mathsf{dABS}\,(\mathsf{dV}\,1)$ is $\lambda x.\,a$, since index 1 under one binder is the free index 0. Under the binder, $\mathsf{inc}(a\;b)$ exchanges the indices 1 and 2, so

$$(a\;b) \cdot \mathsf{dABS}\,(\mathsf{dV}\,1) = \mathsf{dABS}\,(\mathsf{dV}\,2),$$

which is $\lambda x.\,b$. Exchanging 0 and 1 under the binder instead would give $\mathsf{dABS}\,(\mathsf{dV}\,0)$, which is $\lambda x.\,x$.

A relation R on dB is *equivariant* if $R\,(\pi \cdot t)\,(\pi \cdot u) \iff R\,t\,u$ for all π.

By [13, Theorem 10], sub and dLAM commute with permutations, which act on their index arguments j as $\widehat{\pi}(j)$, and hence $\to_{\mathrm{d}'}$ is equivariant.

The key observation of [13] is that lifting is itself the action of a permutation [13, Lemma 14], and so is the substitution of a fresh index. A dABS term and its presentation as a dLAM term therefore differ by a permutation, which an equivariant relation cannot see.

Theorem 4.14 ([13, Theorem 15])**.** *Let R be an equivariant relation on* dB*. Then*

$$\big(\forall t\,u.\; R\,t\,u \Rightarrow R\,(\mathsf{dABS}\,t)\,(\mathsf{dABS}\,u)\big) \iff \big(\forall i\,t\,u.\; R\,t\,u \Rightarrow R\,(\mathsf{dLAM}\,i\,t)\,(\mathsf{dLAM}\,i\,u)\big).$$

Theorem 4.14 applies to $\to_{\mathrm{d}'}$, which is equivariant. To apply it to $\to_\mathrm{d}$ one needs equivariance of $\to_\mathrm{d}$, and hence the behavior of nsub under a permutation [13, Lemma 17].

Theorem 4.15 ([13, Theorem 19])**.** *For all $t, u \in$ dB: $t \to_{\mathrm{d}'} u$ if and only if $t \to_\mathrm{d} u$.*

Example 4.16 (Running example, continued)**.** For distinct atoms a, b, c, the step $(\lambda a.\,[a\,b]_\alpha)\,[c]_\alpha \to_\beta [c\,b]_\alpha$ becomes

$$\mathsf{dAPP}\,(\mathsf{dABS}\,(\mathsf{dAPP}\,(\mathsf{dV}\,0)\,(\mathsf{dV}\,(\hat{b}+1))))\,(\mathsf{dV}\,\hat{c}) \;\to_\mathrm{d}\; \mathsf{dAPP}\,(\mathsf{dV}\,\hat{c})\,(\mathsf{dV}\,\hat{b})$$

: nsub replaces the bound index 0 by $\mathsf{dV}\,\hat{c}$ and decrements the index of b, whose binder has disappeared.

4.7. Using the isomorphism

An isomorphism of reduction systems transports every property that is stated in terms of the reduction relation. The de Bruijn development we build on proves the Church–Rosser theorem for $\to_{\mathrm{d}}$ [LEAN]. With Theorem 4.6 it becomes a theorem about named terms up to alpha equivalence, with no further work on the named side.

Corollary 4.17 (Church–Rosser for Λ_α)**.** [LEAN] *The relation $\to_\beta$ on Λ_α is confluent: if $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ then there is P with $N_1 \twoheadrightarrow_\beta P$ and $N_2 \twoheadrightarrow_\beta P$.*

Corollary 4.18. [LEAN] *If $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ and both N_1 and N_2 are β-normal, then $N_1 = N_2$.*

The same method applies to η-reduction: f is also an isomorphism between η-reduction on Λ_α and Nipkow's η-reduction on dB [13, Section 4.4] [LEAN].

4.8. Notes on the formalization

- *Atoms.* Sections 2 and 3 are parametric in $\mathbb{A}$. The isomorphism is stated for one concrete type of atoms, the natural numbers wrapped in a structure, for which the two hat maps are the projection and the constructor [LEAN].
- *Permutations.* Norrish and Vestergaard represent a permutation as a list of pairs of strings. The formalization uses finitely supported bijections on atoms [LEAN]. Accordingly `inc` is not defined by recursion over a list but as the permutation that fixes the atom with number 0 and otherwise acts as π with all numbers shifted by one [LEAN].
- *Substitution on* dB. CSLib does not define `nsub` by the equations of Definition 4.1, but as a composition of a lifting, the substitution `sub` and a lowering of indices. The equations of Definition 4.1 are proved as lemmas.
- *The bijection.* f is a bijection in the sense of an explicit pair of mutually inverse functions. Its inverse is obtained from surjectivity by choice and is not computable.
- *Equivariance.* There is no general notion of an equivariant relation in the library used. In Theorem 4.14 equivariance is a hypothesis on R, and it is proved separately for each of the relations to which the theorem is applied.

5. Related work

Mechanized treatments of alpha equivalence as a subject in its own right are few. Crole mentions an Isabelle/HOL development of the results of [5] as work in progress [5, Section 7]; we could find no published account of it. Apinis and Ahman [1] define alpha equivalence of untyped lambda terms as an inductive relation in the Rocq Prover and show that it conforms to the specification given in the literature. Their subject is one definition and its adequacy, whereas Theorem 3.4 compares definitions that the literature itself uses.

The isomorphism of Section 4 was first mechanized by Norrish and Vestergaard in HOL4 [13], and Figure 1 shows the further relationships between formalizations of the lambda calculus that they list.

More broadly, several general-purpose approaches to variable binding exist. Nominal techniques [15], as implemented in Nominal Isabelle [17], provide binder-generic reasoning principles via permutations and freshness. Locally nameless representations [2] aim to make binder-heavy formalizations more tractable. CSLib instead represents named and de Bruijn terms directly as inductive types and proves the relationships between them explicitly, as in Section 4.

6. Future work

The two results of this paper are intended as the first pieces of a larger, ongoing effort within CSLib to build a formalized foundation for research on the untyped lambda calculus. We plan work in three directions.

The network of Figure 1. The alpha equivalence by which Λ_α is formed is $\sim_{\mathrm{p}}$, the same relation by which the HOL4 development underlying [13] quotients its raw terms.[7] By Theorem 3.4, Λ_α is equally the quotient by $\sim_{\mathrm{r}}$ and by $\sim_{\mathrm{r}\#}$, the relations of Barendregt [3, Definition 2.1.11] and of Hindley and Seldin [10, Definition 1.17]. Since substitution on Λ_α is lifted from raw terms (Definition 4.3), the construction is also that of λ_α^{Ho}, which Norrish and Vestergaard describe as a quotient of Curry's raw calculus λ^C with substitution lifted to the quotient [13, Section 2]; the raw terms of Section 2, with the substitution of Definition 2.4, are such a calculus. Homeier's alpha equivalence, however, is a contextual relation defined without swapping or substitution [11, Definitions 9 and 10], so this edge, too, requires identifying it with one of the five. The alpha equivalence of λ^C, as Vestergaard presents it, is the congruence generated by Curry's rule with a fresh atom [18, Section 1.2]. It has the form of $\sim_{\mathrm{r}\#}$ but is stated with Curry's substitution instead of that of Definition 2.4, so identifying the two requires showing that both substitutions agree up to alpha equivalence. The alpha equivalence of λ^V, mechanized by Vestergaard and Brotherston [19], uses a substitution that never renames, together with a side condition that excludes capture. It appears to contain $\sim_{\mathrm{r}}$ and to be contained in $\sim_{\mathrm{r}\#}$, which by Theorem 3.4 would make it equal to all five; we have not yet checked this formally.

Theorem 3.4 thus reduces each remaining α-quotient edge of the figure to identifying the alpha equivalence of a raw calculus with one of the five relations, and the remaining isomorphisms to further representations: Gordon's two-sorted de Bruijn terms dB_2 and Vestergaard's raw calculus λ^V, whose isomorphism with λ^C is mechanized in Coq [13, Section 2.3]. We plan to add these nodes and edges, so that the whole network is formalized within one library.

Further equivalence results. Crole-style results, showing several natural but superficially different definitions of the same notion to coincide, are not unique to alpha equivalence. We plan analogous formalizations for the SK combinator calculus and other binding representations (locally nameless terms, nominal terms), each intended to slot into CSLib alongside the results of this paper and to be connected to them by explicit isomorphisms in the style of Section 4.

Teaching. Fabrizio Montesi (University of Southern Denmark) and Alexandre Rademaker (Getulio Vargas Foundation, Rio de Janeiro), both CSLib maintainers, have expressed interest in using the named representation of Section 2 to teach the lambda calculus and to base assignments on it. It suits this purpose because it stays close to textbook presentations: terms with named variables, capture-avoiding substitution, and, through Theorem 3.4, definitions of alpha equivalence found in the literature [3, 10], all available side by side and proved equivalent. Supporting such use, with documentation and examples aimed at students, is part of our plans for the library.

7. Conclusion

We have formalized in Lean 4, as part of CSLib, Crole's result that five definitions of alpha equivalence coincide and, following Norrish and Vestergaard, an isomorphism between the quotient of named terms by alpha equivalence and pure de Bruijn terms. Together these results connect two of the standard approaches to variable binding within a single library. They are meant to be maintained as part of CSLib rather than as a standalone development: subject to code review, kept up to date with CSLib and Mathlib, and structured so that later projects can depend on them directly. They are the starting point of a foundation for research on the untyped lambda calculus, to be extended by further binding representations and the remaining relationships of Figure 1, and to serve in teaching the lambda calculus.

Data availability

The Lean 4 formalization discussed in this paper is developed for CSLib (`https://github.com/leanprover/cslib`). The parts that are not yet merged there are under review in the pull requests cited in the text, and available at `https://github.com/chris-anto-froeschl/cslib`, at the revisions that the links in this paper point to. A blueprint of both formalizations, with a dependency graph showing their connections, is at `https://chris-anto-froeschl.github.io/untyped-lambda-calculus/`.

[7] Rule `aeq_lam` in `examples/lambda/basics/generic_termsScript.sml` of the HOL4 sources.

CRediT authorship contribution statement

Chris Anto Fröschl: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Visualization. **Matthias Güdemann:** Conceptualization, Supervision, Writing – review and editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to transcribe mathematical notation from the PDF versions of the cited papers into LaTeX, to check the text for errors and grammar, and to reconstruct Figure 1 from [13, Figure 1]. All transcriptions were checked against the originals. AI assistance also helped to locate less visible sources: the HOL4 development behind the proof pearl [13], and earlier sources for the lemmas of Section 6 of [5]. All of them were checked by the authors against the original texts. The authors take full responsibility for the content of the published article.

References

[1] Apinis, K., Ahman, D., 2025. A simple formalization of alpha-equivalence. doi:10.48550/arXiv.2507.10181, arXiv:2507.10181.

[2] Aydemir, B., Charguéraud, A., Pierce, B.C., Pollack, R., Weirich, S., 2008. Engineering formal metatheory, in: Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '08), Association for Computing Machinery, New York, NY, USA. pp. 3–15. doi:10.1145/1328438.1328443.

[3] Barendregt, H.P., 1984. The Lambda Calculus: Its Syntax and Semantics. volume 103 of *Studies in Logic and the Foundations of Mathematics*. Revised ed., North-Holland.

[4] Barrett, C., Chaudhuri, S., Montesi, F., Grundy, J., Kohli, P., de Moura, L., Rademaker, A., Yingchareonthawornchai, S., 2026. CSLib: The Lean computer science library. doi:10.48550/arXiv.2602.04846, arXiv:2602.04846.

[5] Crole, R.L., 2012. Alpha equivalence equalities. Theoretical Computer Science 433, 1–19. doi:10.1016/j.tcs.2012.01.030.

[6] Curry, H.B., Feys, R., 1958. Combinatory Logic. volume I. North-Holland, Amsterdam.

[7] Gabbay, M.J., 2011. Foundations of nominal techniques: logic and semantics of variables in abstract syntax. Bulletin of Symbolic Logic 17, 161–229. doi:10.2178/bsl/1305810911. numbering follows the author version, https://www.gabbay.org.uk/papers/fountl.pdf.

[8] Gabbay, M.J., Mathijssen, A., 2008. Capture-avoiding substitution as a nominal algebra. Formal Aspects of Computing 20, 451–479. doi:10.1007/s00165-007-0056-1.

[9] Gabbay, M.J., Pitts, A.M., 2002. A new approach to abstract syntax with variable binding. Formal Aspects of Computing 13, 341–363. doi:10.1007/s001650200016.

[10] Hindley, J.R., Seldin, J.P., 2008. Lambda-Calculus and Combinators: An Introduction. Second ed., Cambridge University Press. doi:10.1017/CBO9780511809835.

[11] Homeier, P.V., 2001. A proof of the Church-Rosser theorem for the lambda calculus in higher order logic, in: Boulton, R.J., Jackson, P.B. (Eds.), TPHOLs 2001: Supplemental Proceedings, Division of Informatics, University of Edinburgh. pp. 207–222. Informatics Research Report EDI-INF-RR-0046.

[12] Nipkow, T., 2001. More Church–Rosser proofs (in Isabelle/HOL). Journal of Automated Reasoning 26, 51–66. doi:10.1023/A:1006496715975.

[13] Norrish, M., Vestergaard, R., 2007. Proof pearl: De Bruijn terms really do work, in: Schneider, K., Brandt, J. (Eds.), Theorem Proving in Higher Order Logics, Springer, Berlin, Heidelberg. pp. 207–222. doi:10.1007/978-3-540-74591-4_16.

[14] Pitts, A.M., 2006. Alpha-structural recursion and induction. Journal of the ACM 53, 459–506. doi:10.1145/1147954.1147961.

[15] Pitts, A.M., 2013. Nominal Sets: Names and Symmetry in Computer Science. volume 57 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press. doi:10.1017/CBO9781139084673.

[16] Urban, C., Pitts, A.M., Gabbay, M.J., 2004. Nominal unification. Theoretical Computer Science 323, 473–497. doi:10.1016/j.tcs.2004.06.016.

[17] Urban, C., Tasson, C., 2005. Nominal techniques in Isabelle/HOL, in: Nieuwenhuis, R. (Ed.), Automated Deduction – CADE-20, Springer, Berlin, Heidelberg. pp. 38–53. doi:10.1007/11532231_4.

[18] Vestergaard, R., 2003. Structural induction and the λ-calculus. RIMS Kôkyûroku 1318, 30–45. URL: https://www.kurims.kyoto-u.ac.jp/~kyodo/kokyuroku/contents/pdf/1318-5.pdf.

[19] Vestergaard, R., Brotherston, J., 2003. A formalised first-order confluence proof for the λ-calculus using one-sorted variable names. Information and Computation 183, 212–244. doi:10.1016/S0890-5401(03)00023-3.